Graph Neural Network-based RL Agent for Conquest

The agent is a single Graph Neural Network (GNN) with multiple feed forward decision heads, all trained by self-play with Proximal Policy Optimization (PPO). The shared GNN embeds the board and the game state. Every decision the game asks for is relayed to one of ten decision heads that turn the embeddings into a probability distribution over the legal options. A separate value head (the critic) estimates each player's expected final reward and serves as the baseline for PPO. The network has about 129k trainable weights.

1 State encoding

The board is a graph of 14 territories with fixed connections between them. The state is always encoded from the point of view of the player who is making the move (egocentric view) so the network does not have to learn the same strategy separately for different seats. Players are one-hot encoded into seat slots by turn order: slot 1 is always the deciding player, slot 2 the player who acts next, and so on, up to 5 slots. The feature vector of each territory i becomes:

$$x_i = \Big[\underbrace{o_{i,1}, \ldots, o_{i,5}}_{\text{owner (one-hot)}}, \underbrace{u_i}_{\text{unclaimed}}, \underbrace{n_{i,1}, \ldots, n_{i,5}}_{\text{units per owner}} \Big] \in \mathbb{R}^{11},$$

where $o_{i,k} = 1$ if slot k owns territory i, $u_i = 1$ if the territory is unclaimed, and $n_{i,k}$ is the territory's unit count divided by 10 in its owner's slot and 0 in the other slots. Putting units in the owner's column means that summing aggregation over neighbours keeps troop totals separate for each player. The resulting x_i feature vectors are packed into a matrix representing the board state:

$$X = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_{14} \end{bmatrix} \in \mathbb{R}^{14 \times 11}$$

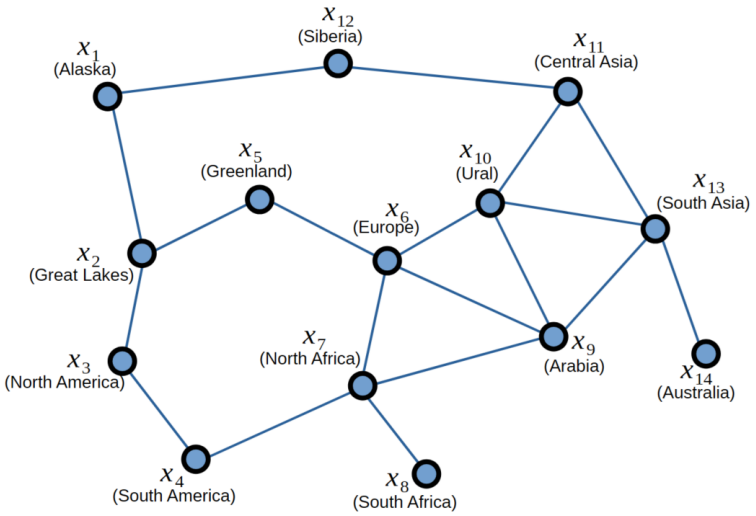


Figure 1: Graph of the board encoded by the adjacency matrix $\hat{M}$

2 GNN embedding

Each territory's encoded feature vector is embedded in a 107-dimensional space through three message-passing (GNN) layers and concatenation. In every layer, the sum of the neighbours' features is concatenated to the node's features from the previous layer and the resulting vector is projected, shifted and passed through a ReLU activation function:

$$H_0 = X, \qquad H_{l+1} = \mathrm{ReLU}\left(\left[\, H_l \,\middle\|\, \hat{M} H_l \,\right] W_l^{\mathrm{T}} + \mathbf{1} \cdot b_l^{\mathrm{T}}\right), \qquad l = 0, 1, 2,$$

$$H_{l+1} = \mathrm{ReLU}\left(\begin{array}{|c|c|} \hline H_l & MH_l \\ \hline \end{array} \begin{array}{|c|} \hline W_{l,\mathrm{self}}^{\mathrm{T}} \\ \hline W_{l,\mathrm{neighb}}^{\mathrm{T}} \\ \hline \end{array} + \begin{array}{|c|} \hline 1 \\ 1 \\ \vdots \\ 1 \\ 1 \\ \hline \end{array} \begin{array}{|c|} \hline b_l^{\mathrm{T}} \\ \hline \end{array} \right)$$

where $\|$ denotes concatenation, $\hat{M}$ is the board's adjacency matrix (without self-loops, the diagonal is zero), W_l ($W_0 \in \mathbb{R}^{22\times 32}, W_1, W_2 \in \mathbb{R}^{64\times 32}$) and $b_l \in \mathbb{R}^{32}$ are trainable weights and biases, and $\mathbf{1} \in \mathbb{R}^{14}$ is a column vector of ones. The territory's own features have their own weights, so they are not diluted by its number of neighbours and the sum keeps totals: in the first layer, the neighbourhood message counts how many neighbours each player owns and how many units they hold there. The final embedding concatenates the input and all three layer outputs and contains the raw state plus information from 1, 2 and 3 hops away:

$$E = \left[\, X \,\|\, H_1 \,\|\, H_2 \,\|\, H_3 \,\right] \in \mathbb{R}^{14\times 107}.$$

This architecture is esentially a GraphSAGE with summing every neighbour, instead of sampling them.

3 Decision heads

A turn is broken into a sequence of single decisions over the set of legal options. For follow-up decisions in a multi-step action (e.g. attack from → attack to → continue attack?), the embeddings of the previously selected territories s_1, s_2, and the unit count divided by 10 where applicable, are appended to every territory's row as context forming the augmented matrix

$$\tilde{E} = [\, E \,\|\, E_{s_1} \,\|\, E_{s_2} \,\|\, V \,],$$

where

$$E_{s_1} = \begin{bmatrix} h_{s_1} \\ h_{s_1} \\ \vdots \\ h_{s_1} \end{bmatrix}, \qquad E_{s_2} = \begin{bmatrix} h_{s_2} \\ h_{s_2} \\ \vdots \\ h_{s_2} \end{bmatrix}$$

are the embedded feature vectors of the selected territories repeated in a matrix and

$$V = \begin{bmatrix} v \\ v \\ \vdots \\ v \end{bmatrix} \in \mathbb{R}^{14\times 1}$$

is the constant vector of the normalized unit count. The augmented state matrix $\tilde{E}$ is passed through one linear layer compressing the feature vectors, followed by a ReLU activation:

$$D = \mathrm{ReLU}\left(\tilde{E} \cdot W_3^{\mathrm{T}} + \mathbf{1} \cdot b_3^{\mathrm{T}}\right) \in \mathbb{R}^{14\times 32}$$

The ReLU nonlinearity serves two purposes here:

- It ensures that the appended identical vectors do contribute differently to each territory's row. Without a nonlinearity, a context column appended identically to every row would collapse into a constant bias on the logits. The ReLU makes the shared context interact with each territory's own embedding, so it contributes differently to each row.
- It also compresses rows to 32 features before flattening, keeping the output layer small (448 inputs instead of 3000). The final layer provides full-board vision by flattening the state matrix into a vector.

Then the 14 rows are flattened into one 448-dimensional vector:

$$D \in \mathbb{R}^{14\times 32} \ \rightarrow D_{\mathrm{vec}} \in \mathbb{R}^{1\times 448}$$

and a linear layer gives one score (logit) per option:

$$r = D_{\mathrm{vec}} \cdot W_4^{\mathrm{T}} + b_4^{\mathrm{T}}$$

Each output has its own weights for every territory and feature, so the heads can learn territory-specific strategy. Illegal options get $r_j = -\infty$ (masking) before the softmax, so illegal options have 0 probability:

$$\pi(j) = \mathrm{softmax}(r)_j$$

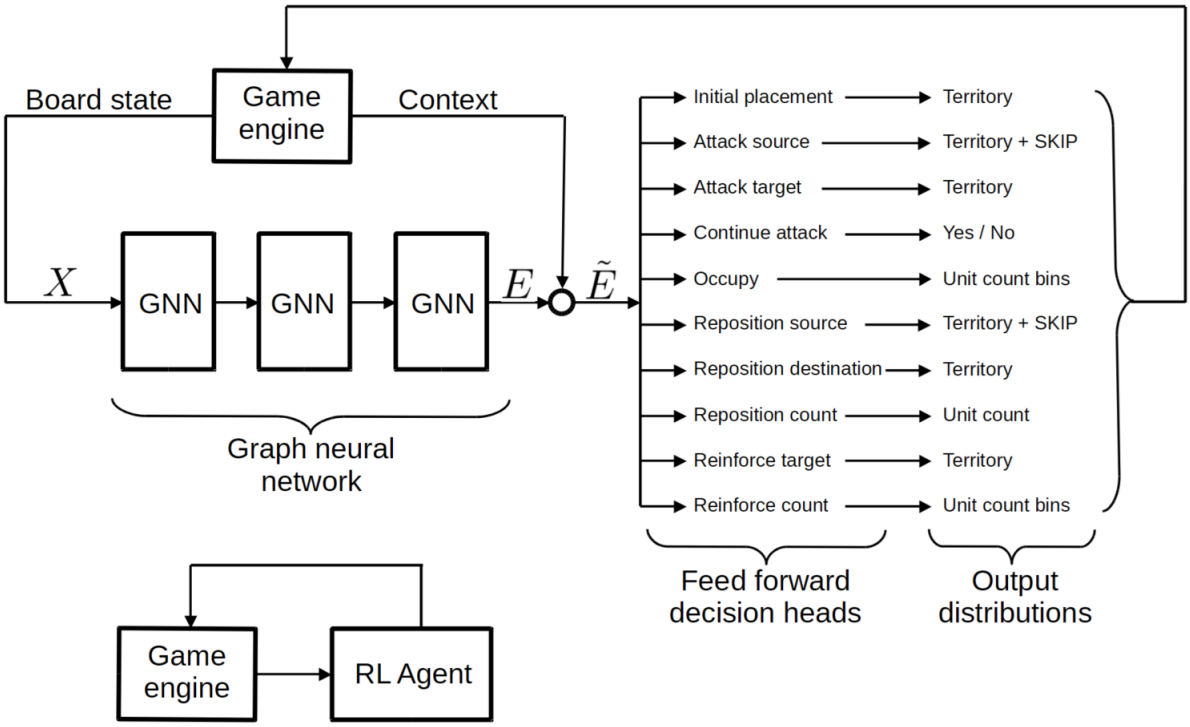
Unit count heads output 11 bins spread evenly over the legal range $[m, M]$:

$$\mathrm{count}(k) = m + \mathrm{round}\Big(\frac{k}{10}\,(M-m)\Big), \qquad k = 0, \ldots, 10.$$

Every bin is legal, and bin 10 always means "everything", whatever the stack size.

All decision heads share the same structure with different W_3, W_4, b_3, b_4 weights and biases, only the input and output sizes differ:

Decision	Options	Context appended to each row	Input and output sizes
initial placement	14 territories	placements left	$14 \times 108 \rightarrow 14$
attack source	14 territories + SKIP	—	$14 \times 107 \rightarrow 15$
attack target	14 territories	source	$14 \times 214 \rightarrow 14$
continue attack	stop / continue	source, target	$14 \times 321 \rightarrow 2$
occupy	11 count bins	source, conquered territory	$14 \times 321 \rightarrow 11$
reposition source	14 territories + SKIP	—	$14 \times 107 \rightarrow 15$
reposition destination	14 territories	source	$14 \times 214 \rightarrow 14$
reposition count	11 count bins	source, destination	$14 \times 321 \rightarrow 11$
reinforce target	14 territories	reinforcements left	$14 \times 108 \rightarrow 14$
reinforce count	11 count bins	target, reinforcements left	$14 \times 215 \rightarrow 11$



During training, actions are sampled from π; in play, the most likely option is taken. Forced decisions (only one legal option) bypass the network and are not learned from.

4 Value head (critic)

The critic has the same per-territory linear layer, flatten and output structure as the other heads, but with a single output $V(s)$: the expected final reward for the deciding player. At its input, each territory's row is its embedding plus seven context columns:

$$Z_V = \big[\, E \,\|\, \text{phase one-hot columns (4)} \,\|\, \text{units still to place column} \,\|\, e_{s_1} \,\|\, e_{s_2} \,\big].$$

- The four phases are expansion, combat, repositioning and reinforcement.
- "Units still to place" means placements left or reinforcements left, and 0 otherwise.
- The indicator columns e_{s_1} and e_{s_2} mark the territories already chosen in the current move. Unlike the policy context, the value context uses one-hot markers for previous selections.
- Together these tell the critic where in the turn the decision happens, which the board alone cannot show.

The same critic also produces the position values in the console legend.

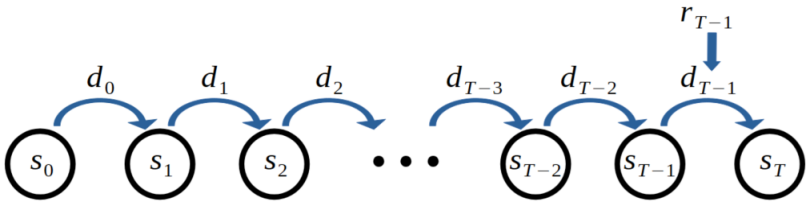
5 Proximal Policy Optimisation (PPO)

The network is trained by self-play. In every iteration, a fixed number of games are played with a random number of players, with the same agent playing every seat. Each player's sequence of decisions at $t \in \{0, \ldots, T-1\}$ in each game are collected into trajectories. Everything the opponents do between steps is part of the environment. The only reward, $r_{T-1} = R$, is for the last decision of each trajectory: Players get $+1$ for the last decision leading to a win and -1 for a loss. For games stopped at 60 rounds, players already eliminated receive -1 and survivors get their territory share minus 0.5. This makes the game zero-sum for two players, but non-zero-sum for more than two players, correctly reflecting the game dynamics.

During the iteration's self-play games the weights θ_{old} don't change and a_t is sampled from the policy distribution $\pi_{\theta_{\text{old}}}(\cdot \mid s_t)$. Decisions are stored in the trajectories as tuples of the form

$$d_t = \{s_t,\, a_t,\, \log \pi_{\theta_{\text{old}}}(a_t \mid s_t),\, V_{\theta_{\text{old}}}(s_t)\},$$

where s_t is the board state (including valid options, head-ID and context, where needed) in which the decision was made, a_t is the index of the action taken, $\log \pi_{\theta_{old}}(a_t \mid s_t)$ is the log-probability of taking action a_t in state s_t with network parameters θ_{old}, and $V_{\theta_{\text{old}}}(s_t)$ is the critic value of state s_t with critic parameters θ_{old}.



Rewards are propagated backward through the trajectory with Generalised Advantage Estimation (GAE):

$$A_t = \sum_{k=0}^{T-1-t} (\gamma\lambda)^k \delta_{t+k}$$

calculated recursively as

$$A_t = \delta_t + \gamma\lambda\, A_{t+1}, \quad A_T = 0$$

where δ_t are the temporal difference errors, in this case

$$\begin{aligned} \delta_t &= \gamma\, V_{\theta_{old}}(s_{t+1}) - V_{\theta_{old}}(s_t), && \text{for} \quad t \in \{0, \ldots, T-2\} \\ \delta_{T-1} &= R - V_{\theta_{old}}(s_{T-1}), && \text{for} \quad t = T-1 \end{aligned}$$

with discount and GAE parameters γ and λ. For the policy loss, the advantages are normalised to mean 0 and standard deviation 1 over the iteration's data, giving:

$$\hat{A}_t = \frac{A_t - \mu_A}{\sigma_A + 10^{-8}}$$

This keeps the policy step size independent of the reward scale. Only the policy loss uses $\hat{A}$; the critic's targets R_t use the raw advantages. The policy loss is the PPO clipped policy ratio:

$$L_P = -\min\big(\rho_t \hat{A}_t,\ \mathrm{clip}(\rho_t, 1-\varepsilon, 1+\varepsilon)\,\hat{A}_t\big)$$

The clipping removes any further incentive to increase an action's probability more than ε (20%) from θ_{old} within one update. The advantages were measured under θ_{old} and only stay trustworthy near it. The negative sign ensures the optimiser maximises the expected reward with gradient ascent. The probability ratio

$$\rho_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)} = \exp\big(\log \pi_\theta(a_t \mid s_t) - \log \pi_{\theta_{\text{old}}}(a_t \mid s_t)\big)$$

compares the current policy with the policy that played the games, describing how much more (or less) likely the stored action is now than when it was played. The code computes the ratio from the difference of log-probabilities, which is numerically safer than dividing small probabilities. On the very first minibatch the weights haven't changed yet, so $\rho = 1$ everywhere.
The critic learns towards

$$R_t = A_t + V_{\theta_{\text{old}}}(s_t),$$

and the per-sample loss is

$$L_V = \big(R_t - V_\theta(s_t)\big)^2 = \big(A_t + V_{\theta_{\text{old}}}(s_t) - V_\theta(s_t)\big)^2,$$

where R_t was recorded with the old weights θ_{old} and does not change during the iteration.

The total loss of the system over a minibatch B (512 decisions in the current setup) is

$$L(\theta) = \frac{1}{|B|}\sum_{t\in B}\Big[-\min\big(\rho_t \hat{A}_t,\ \mathrm{clip}(\rho_t, 1-\varepsilon, 1+\varepsilon)\,\hat{A}_t\big) + c_v \cdot \tfrac{1}{2}\big(R_t - V_\theta(s_t)\big)^2 - c_e\, H\big[\pi_\theta(\cdot \mid s_t)\big]\Big],$$

i.e. the clipped policy ratio, the value loss and an entropy bonus. The entropy bonus

$$H\big[\pi_\theta(\cdot \mid s_t)\big] = -\sum_j \pi_\theta(j \mid s_t)\log \pi_\theta(j \mid s_t)$$

is subtracted to incentivise exploration during training and prevent the policy from prematurely collapsing into a local optimum. One iteration's data is used up to E passes (epochs), each with a fresh shuffle, in minibatches of $|B|$ decisions and Adam optimiser with a per-iteration learning-rate decay of 0.98.

Inside each head, let z denote the masked logits and $\pi = \mathrm{softmax}(z)$ the resulting distribution over legal options. The gradient of the chosen action's log-probability with respect to each logit is then

$$\frac{\partial \log \pi_a}{\partial z_j} = \mathbb{1}[j = a] - \pi_j.$$

So a positive advantage raises z_a and lowers every other logit in proportion to its probability. A masked option has $\pi_j = 0$, and its gradient is blocked anyway. Its logit is never touched.

From there, autograd propagates the gradient back through the output layer W_4 the compression layer W_3, the tiled context vectors (E_{s_1}, E_{s_2}, V, built by gather and expand) and finally the GNN layers W_0, W_1, W_2 updating the shared trunk for all heads at once. To evaluate a minibatch, '_evaluate' runs each head once on the rows of its own decision type and scatters the per-row log-probabilities and entropies back into batch-aligned vectors with 'index_put'.

After every iteration, the greedy policy plays 50 two-player games against a uniformly random agent, in a random seat. Games that hit the round cap count as losses. The checkpoint is saved at the same time. The default run is 200 iterations, about 6,400 games. The checkpoint stores the weights, the optimizer state, the PPO settings, the map, and a description of the network layout; a checkpoint is refused if the layout does not match the current code.

Symbol	Meaning	Value
γ	discount per decision	0.999
λ	GAE parameter	0.95
ε	ratio clipping	0.2
c_v	value loss weight	0.5
c_e	entropy bonus weight	0.01
η	Adam learning rate	$3 \cdot 10^{-4}$
E	passes per update	4
$\lvert B \rvert$	minibatch size	512
$g_{\max}$	gradient norm limit	0.5
$\mathrm{KL}_{\mathrm{target}}$	early-stopping threshold	0.03
—	games per iteration	32
—	iterations (default run)	200
—	round cap per game	60
—	unit normalisation divisor	10

Table 1: Hyperparameters.